

Introduction to Object-Oriented Software Construction

Experiment Guidebook

Experiment 3: Strategy Pattern

Harbin Institute of Technology (Shenzhen)

2025

1. Objective

- (1) Understand the motivation and design intent of the Strategy Pattern
- (2) Master the structure of the Strategy Pattern and be able to draw its UML class diagram
- (3) Refactor existing code and implement the Strategy Pattern in practice

2. Environment

- Operating System: Windows 10
- IDE: IntelliJ IDEA 2025.2
- JDK: OpenJDK 24 or higher
- Tools: PlantUML plugin for IntelliJ IDEA

3. Experiment Tasks

(1) **Design Pattern Application:** Identify scenarios in the game that are suitable for the **Strategy Pattern**—specifically, the implementation of different shoot trajectories and firepower power-up effects. Draw the corresponding UML class diagram, then refactor the codebase to apply the Strategy pattern, thereby improving extensibility and maintainability.

(2) **Iterative Development Goal:** two types of firepower power-ups take effect

We will implement two types of firepower power-ups:

- ✧ **Firepower Supply**  (from the original requirements): When the hero aircraft collects this power-up, its bullet trajectory changes from the original one to a spread shot, firing 5 bullets at once in a fan-like formation.
- ✧ **Super Firepower Supply**  (a new requirement): When the hero aircraft picks up it, the bullet trajectory switches to a circular pattern (firing 20 bullets simultaneously)—for example, changing from straight shot to circular shot. You may also design your own custom bullet trajectory for this behavior.

4. Experiment Steps

4.1 Strategy Pattern

The **Strategy Pattern** is a behavioral design pattern that enables an algorithm's behavior to be selected at runtime. It defines a family of **interchangeable algorithms**, encapsulates each one as a separate class, and makes them all conform to a common interface. In this way, the context (i.e., the object that uses the algorithm) can then switch between these algorithms dynamically without changing its own code.

Key points of the Strategy Pattern:

✧ **Encapsulation & Interchangeability of Algorithms**

Define a Strategy Interface to unify encapsulation of different algorithms / behaviors. Each algorithm maps to a Concrete Strategy Class that implements the interface. A Context Class holds a Strategy interface reference, delegates behavior execution to the strategy object (no direct dependency on concrete algorithms).

✧ **Runtime Dynamic Switching**

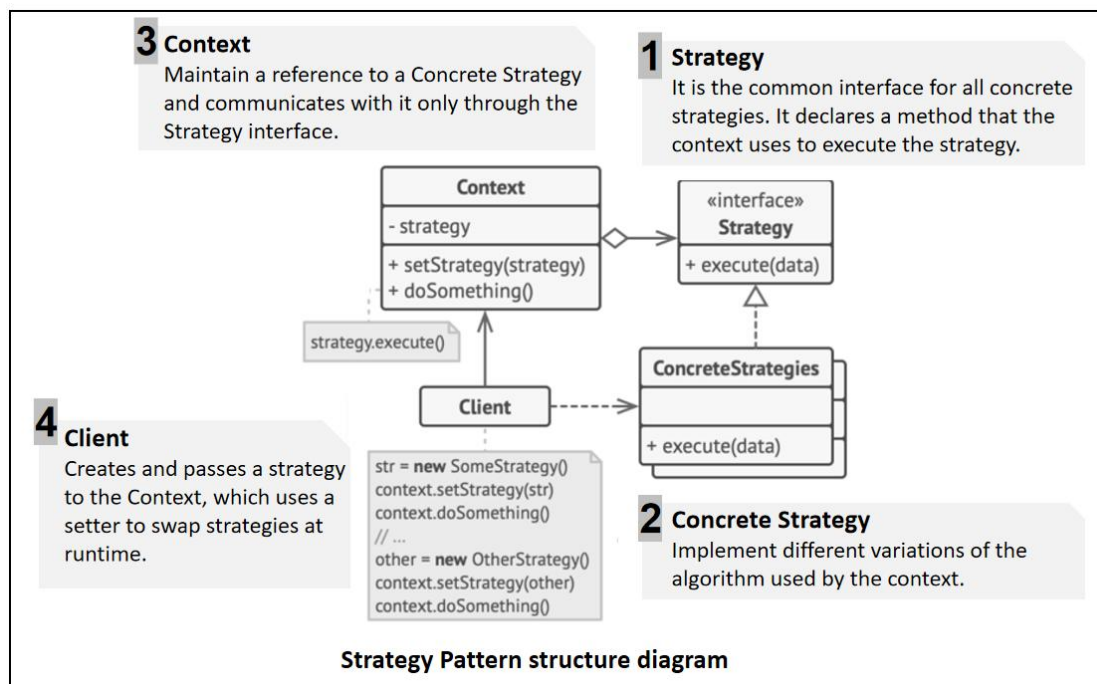
The Context changes its behavior by replacing the concrete strategy object it holds. This switch is transparent to the client, ensuring low coupling and high cohesion.

✧ **Compliance with Open/Closed Principle**

Add new algorithms by creating new Concrete Strategy classes—no modifications to the Context or existing strategies needed. Avoids redundant if-else/switch statements, reducing complexity and maintenance costs.

✧ **Compliance with Composite Reuse Principle**

The Context extends behavior via composition (holding a strategy reference) instead of inheritance, offering far greater flexibility.



Tasks to be completed:

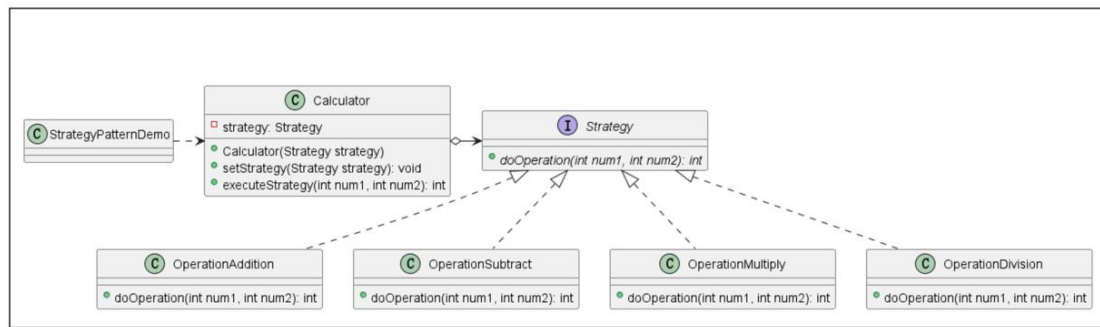
(1) Please refer to the above Strategy Pattern structure diagram to **draw the Strategy pattern UML class diagram** for aircrafts and different bullet trajectories in this game. **The client side does not need to be drawn.** Save the diagram in the uml directory.

(2) **Refactor the code** based on the Strategy Pattern you designed to implement **three types of bullet trajectories**: straight shot, spread shot and circular shot. Ensure that the hero aircraft and the three types of enemy aircraft use the Strategy Pattern to implement bullet trajectory.

4.2 Strategy Pattern Application Example

Suppose we want to implement a calculator that supports four operations: addition, subtraction, multiplication, and division. How can we apply the Strategy Pattern to achieve this?

The following is the UML class diagram design for applying the Strategy Pattern to our calculator problem.



Code examples of the Strategy Pattern :

We will create a Strategy interface and four concrete strategy classes that implement this interface. The Calculator class will act as the Context role; it uses a specific strategy. The StrategyPatternDemo (our Client) will utilize the Calculator and these strategy objects to demonstrate how the behavior of the Calculator (the Context) changes dynamically when its configured strategy is replaced at runtime.

Step 1: Create an interface to act as the abstract Strategy role

Strategy.java

```
public interface Strategy {
    int doOperation(int num1, int num2);
}
```

Step 2: Create four concrete strategy classes (Addition, Subtraction, Multiplication, Division) that implement the Strategy interface.

OperationAdd.java

```
public class OperationAdd implements Strategy{
    @Override
    public int doOperation(int num1, int num2) {
        return num1 + num2;
    }
}
```

OperationSubtract.java

```
public class OperationSubtract implements Strategy{
    @Override
    public int doOperation(int num1, int num2) {
        return num1 - num2;
    }
}
```

OperationMultiply.java

```
public class OperationMultiply implements Strategy{
    @Override
    public int doOperation(int num1, int num2) {
        return num1 * num2;
    }
}
```

OperationDivision.java

```
public class OperationDivision implements Strategy{
    @Override
    public int doOperation(int num1, int num2) {
        try {
            int num3 = num1 / num2;
        } catch (Exception e) {
            e.printStackTrace();
        }
        return num1 / num2;
    }
}
```

Step 3: Create Calculator class to act as the Context role

Calculator.java

```
public class Calculator{
    private Strategy strategy;

    public Calculator(Strategy strategy){
        this.strategy = strategy;
    }

    public void setStrategy(Strategy strategy) {
        this.strategy = strategy;
    }

    public int executeStrategy(int num1, int num2){
        return strategy.doOperation(num1, num2);
    }
}
```

Step 4: Use StrategyPatternDemo to demonstrate how the calculator's behavior changes when its strategy is switched

StrategyPatternDemo.java

```
public class StrategyPatternDemo {  
    public static void main(String[] args) {  
  
        Calculator calc= new Calculator(new OperationAddition());  
        System.out.println("10 + 5 = " + calc.executeStrategy(10, 5));  
  
        calc.setStrategy(new OperationSubtract());  
        System.out.println("10 - 5 = " + calc.executeStrategy(10, 5));  
  
        calc.setStrategy(new OperationMultiply());  
        System.out.println("10 * 5 = " + calc.executeStrategy(10, 5));  
  
        calc.setStrategy(new OperationDivision());  
        System.out.println("10 / 5 = " + calc.executeStrategy(10, 5));  
    }  
}
```

Step 5: Execute the program and output the results

```
10 + 5 = 15  
10 - 5 = 5  
10 * 5 = 50  
10 / 5 = 2
```

The output shows different results. Thus, even though we're using the same code structure, different strategies produce different calculation outcomes.

That's **polymorphism**: the same interface, different behaviors.

4.3 Two Types of Firepower Power-ups

In the original requirements, we only had one type of firepower power-up. Now, we need to make a **requirement change** to add a new type of firepower power-up. Both of these power-ups can change the hero aircraft's original bullet trajectory to a new specific trajectory. Specifically, the **Firepower Supply** will change the hero aircraft's trajectory to **spread shot**, while the **Super Firepower Supply** will change it to **circular shot**.



Tasks to be completed:

The version submitted in this experiment must include and correctly implement the following functions:

- When the hero aircraft collects a Firepower Supply, its bullet trajectory immediately switches to a spread pattern—for example, from straight shot to firing 5 bullets at once in a fan-like formation.
- When the hero aircraft collects a Super Firepower Supply, the trajectory changes to a circular pattern, firing 20 bullets simultaneously in all directions.

Note: This lab does not involve multithreading. Once the trajectory changes, it won't revert back. Full support for switching and restoring behaviors will be added in Lab 4.

Reference code for spread fire trajectory:

```
public List<BaseBullet> shoot() {
    List<BaseBullet> res = new LinkedList<>();
    int x = this.getLocationX();
    int y = this.getLocationY() + direction*2;
    int speedX = this.getSpeedX();
    int speedY = this.getSpeedY() + direction*5;
    BaseBullet bullet;
    for(int i=0; i<shootNum; i++){
        bullet = new EnemyBullet(x + (i * 2 - shootNum + 1) * 10, y,
                                speedX + (i * 2 - shootNum + 1), speedY, power);
        res.add(bullet);
    }
    return res;
}
```


5. Assignment Submission

Submission requirements include:

(1) Project Code (the entire project folder "AircraftWar-base")

- UML class diagrams drawn using the PlantUML plugin (saved in the uml directory)
- Fully functional and runnable code

(2) Experiment Report

Please use the provided report template

Place both the project code and the lab report in the same folder, then compress the entire folder into a single archive file (.zip) for submission.

Deadline:

Submit your work to the **HITsz Grader** assignment submission platform **within one week** after the lab session. The exact due date will be specified on the platform.

Login URL: <https://grader.cs-lab.top/>

The default username and password are your student ID.

Note: After uploading, please download your submission to verify that it was uploaded successfully.